# LESSON 08

# 每遍 记账




## 累加和计数：让循环帮你记账

学完你能让程序算总分、算平均分，还能数出有几个人及格。

# 全班总分怎么算?

第一次

**投 5 元**

第二次

**投 10 元**

第三次

**投 3 元**

存钱罐里

**18 元**

循环每一遍读到一个分数，下一遍就换成新的了，前面的数留不住。

办法就像存钱罐：放一个变量在循环外面，每一遍往里加一点，到最后再打开看。

# 累加：一遍加一点

sum_three.py

```python
total = 0
for i in range(1, 4):
    total = total + i
print(total)
```

循环结束，输出 total，也就是 1 + 2 + 3 的和。

| | |
|---|---|
| i = 1 | 1 |
| i = 2 | 3 |
| i = 3 | 6 |
| 输出 | 6 |

# 猜一猜：放错位置会怎样?

**total = 0 放进循环**

```
for i in range(1, 4):
    total = 0
    total = total + i
print(total)
```

**3**

**print 放进循环**

```
total = 0
for i in range(1, 4):
    total = total + i
    print(total)
```

**1 3 6**

左边每一遍都把存钱罐倒空，只留下最后的 3。

右边每加一次就输出一次，一共三行，每个数占一行。

# 累加要放对三个位置

## 1

### 循环前

`total = 0`

准备好，只做一次

## 2

### 循环里

`total = total + i`

每一遍加一点

## 3

### 循环后

`print(total)`

全部加完再输出

看代码的缩进，就能看出每一行在哪个位置。

# 小测：这段程序输出几？

输出几？

```
1 total = 0
2 for i in range(1, 5):
3     total = total + i
4 print(total)
```

1. 10
2. 4
3. 6
4. 15

答对了！range(1, 5) 是 1、2、3、4，加起来是 10。

# 更新有个简写：+=

| | |
|---|---|
| `total += i` | 就是 `total = total + i` |
| `count += 1` | 就是 `count = count + 1` |
| `i -= 1` | 就是 `i = i - 1` |

读作“加上”：`total` 加上 `i`。两种写法结果一样，后面都用短的这种。

注意加号在等号前面。

# 动手改一改

读入 n，再改 range，用上 +=。

## 从 1 加到 n

读入一个 0 到 10000 的整数 n，输出 1 + 2 + … + n 的和。相传数学家高斯小时候很快就算出了 1 加到 100。程序比他还快。

输入 100，输出

**5050**

输入 0，输出

**0**

教室电脑：打开 starter/02_gauss.py

# 例题：全班总分

total.py

```python
n = int(input())
total = 0
for i in range(n):
    score = int(input())
    total += score
print(total)
```

这里 `i` 只管数遍数，不用它的值。输出总分 240。

| | |
|---|---|
| 读到 80 | **80** |
| 读到 90 | **170** |
| 读到 70 | **240** |
| 输出 | **240** |

# 动手改一改

只要改最后一行。

## 改成输出平均分

第一行读入人数 n（1 到 100 的整数），接下来 n 行，每行一个 0 到 100 的整数分数。

| 依次输入 | 输出 |
| --- | --- |
| 3<br>80<br>90<br>70 | 80.0 |

教室电脑：打开 starter/04_average.py

# 计数：碰到一个，记一下

count_multiples.py

```python
n = int(input())
count = 0
for i in range(1, n + 1):
    if i % 3 == 0:
        count += 1
print(count)
```

循环结束，输出 3。像画“正”字，碰到一个画一笔。

| | |
|---|---|
| i = 3 | **1** |
| i = 6 | **2** |
| i = 9 | **3** |
| 输出 | **3** |

# 猜一猜：3、6、9 算出来各是几?

累加：加这个数

```
total += i
```

**3 + 6 + 9 = 18**

计数：只加 1

```
count += 1
```

**1 + 1 + 1 = 3**

问“一共多少”用累加，问“有几个”用计数。
两个变量也可以同时用，各记各的。

# 小测：上面的计数程序，输入 2 输出什么？

1 0

2 2

3 什么也不输出

4 报错

答对了！1 和 2 都不是 3 的倍数，count 一直是开始的 0。

# 动手改一改

把累加换成计数，再加一个判断。

## 数一数几个人及格

第一行读入人数 n（0 到 100 的整数），接下来 n 行，每行一个 0 到 100 的整数分数。60 分及以上算及格，输出及格人数。

| 依次输入 5 行 | 输出 |
| --- | --- |
| 4、80、59、60、30 | 2 |

教室电脑：打开 starter/06_count_pass.py

# 这节课常见的三种错误

划线的写法只阅读，不要照抄

| | | |
|---|---|---|
| 忘了写 `total = 0` | **NameError** | 循环前先准备 |
| ~~`total += "90"`~~ | **TypeError** | 读入后用 `int()` |
| ~~`total =+ i`~~ | **不报错** | 写成 `+=` |

要小心：写成 `=+` 不报错，`total` 每次都被换成 `i`，只留下最后一个数。

# 找错：怎么数成 9 个了？

bug.py（只阅读）

```
1 count = 0
2 for i in range(1, 10):
3     if i % 3 == 0:
4         print(i)
5     count += 1
6 print(count)
```

1 到 9 里只有 3 个 3 的倍数，程序却说有 9 个。
点你认为有问题的那一行。

点代码里的一行

# 自己挑战

# 存钱罐

每行读入这次存了多少元（1 到 1000 的整数），读到 0 就停，0 不算一次。
输出一共存了几次，一共多少元。

依次输入

```
5
10
3
0
```

输出

```
存了 3 次，一共 18 元
```

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　不知道要存几次，用 for 还是 while？上节课猜数字是怎么读的？

**提示 2**　要记两件事，就准备两个变量，各记各的。

**提示 3**　试试第一行就输入 0，应该输出：存了 0 次，一共 0 元。

# # 造一个你的小账本

程序一笔一笔地记，最后告诉你总账。记什么，你自己定。

### 先做到

读入几个数，输出它们的总和，再说一句话。

### 选你喜欢的

零花钱、每天步数、游戏得分、读书页数，什么都行。

### 还想更厉害

同时算总和和次数，再算平均，数一数有几次超过目标。

没有标准答案，能运行、你喜欢就好。

# # 和同桌换着玩

## 1

### 换着记账

帮同桌输入几笔，心算验证对不对

## 2

### 说一个喜欢的

告诉同桌，哪一句总结最有意思

## 3

### 提一个想法

一笔都没有的时候，程序会说什么？

听完再改一版，让账本更好用。

# 说得清吗?

| | | |
|---|---|---|
| Q1 | `total = 0` 为什么放在循环前? | 放在循环里，每一遍都会清零。 |
| Q2 | 累加和计数有什么不同? | 累加加这个数，计数只加 1。 |
| Q3 | `print` 为什么放在循环后? | 全部算完，结果才是对的。 |

# 速查卡

| | |
|---|---|
| `total = 0` | 循环前准备，只做一次 |
| `total += i` | 累加：每一遍加上这个数 |
| `count += 1` | 计数：碰到一个加 1 |
| `print(total)` | 循环后输出，不缩进 |
| `total / n` | 平均数：总和除以个数 |

# # 这些后面再学

### 最大值 最小值

下节课：全班谁的分数最高

### sum() len()

Python 自带的求和和数个数

### round()

把平均分保留几位小数

今天程序学会了一边循环一边记账。下节课让它记住最大的那一个。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 python-for-youth

下一课：最大值和最小值